

Group 61

Julius Goldberg (jlg445), Ben Brown (beb249), Elijah Feist (ecf55)

Robot Design and Strategy Overview

Mechanical: The robot sits atop a cage and has two retracting arms. It uses two ____ servos to drive the wheels through a 8:5 gear train and 9g servos for the arms. Utilized the provided servo accessories to improve torque transfer. Frame extended with acrylic for cost efficiency. Did best to utilize the chassis that was given at the start of the project. Included hard stops into the geometry of the arms so that it would never exceed the geometric bounds established by the rules. Caster wheels for stability and better maneuverability pivoting off the two main drive wheels.

Electrical: We utilize an Arduino Uno with a 9V battery and an external 6V battery which passes through a 12V voltage step-up module and an H-Bridge for each drive motor. To keep the design as simple as possible, we utilize the reset button on the Arduino to activate the program. No external sensors - color sensor, line follower, ultrasonic, etc.

Software: The software divides the movement and arm opening/closing into individual functions for ease of implementation (as further elaborated on in Appendix D) and calibration. We are utilizing the timers OCR1A (left wheel), OCR1B (right wheel), OCR2A (left arm), and OCR2B (right arm) to provide independent pwm control to each servo and so we can calibrate the drive motors which were modified to remove their independent pwm processor. We utilize masking to ensure pins are not overwritten between functions but elected not to use interrupts because of the linear design of our robot behavior.

Design Process Reflection

Over the course of the project, the team worked together reliably through design, testing, assembly, and debugging meetings on Wednesday @ 12:00pm and Friday @ 5:00pm per the initial agreement.

When a particular element of the robot blocked progress in other avenues, we would shift our tasks during labs to maintain progress or have the responsible party do work independently so that everyone was prepared at the labs.

No significant roadblocks as we were able to stick to the initial goal of optimizing for speed and simplicity. However, close to the deadline we realized that the 9g servos drew significant current to maintain their position even when the arms were not actively opening and closing. This limited the speed of the driving servo, which is also consuming from the 6V power supply. After moving the servos to the 5V rail from the Arduino Uno, we realized the full potential of the 6V battery and step-up.

Competition Analysis

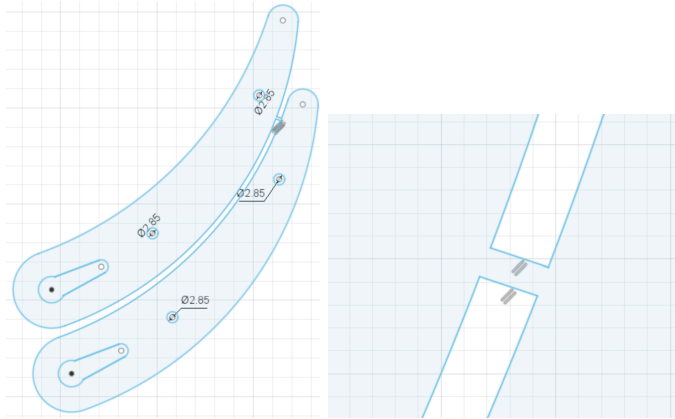
The day of, we had many technical difficulties which limited our performance. Although we'd confirmed that the vehicle would perform its desired path the night before, there were several failures on the day of the competition which limited our actual performance. The success rate of our vehicle was directly predicated on both (a) getting to the blocks before the other robot and (b) collecting at least 50% of the blocks so that we have an ensured victory. However, we struggled to accomplish both of these things. From a design standpoint, the blocks would sometimes get caught in the wheels, altering the trajectory of the hard-coded path. From an electrical/software perspective, we may have been damaging servos based on their inconsistent behavior and noisy outputs even after they've ceased to move or reached a mechanical limit imposed by the design. Finally, electrically we couldn't realize the full potential of the voltage step-up due to insufficient current tolerance we believe. This led our robot to behave inconsistently, and not achieve its objectives. We still won a couple of matches, but not as dominantly as we would have liked, and many times our robot would veer off course and not collect any blocks at all. We finished with a win / loss rate of 3.5 / 4.5 within our division.

Conclusion

The robot was a great learning experience, giving us insights into how to formulate requirements as we're approaching something novel. While the idea of a fast robot is enticing for its simplicity to implement, perhaps it would have been wise to have a backup plan in case the step-up unit didn't work, which was the case. It was also very challenging to get the electronics and software working as intended, particularly because we were dealing with different electrical components than others. Multiple TAs couldn't figure out why we weren't seeing the right voltage output from the step-up unit. Still, we each learned a lot about integrating with different components and engaging with them at a lower level than ever before, which was incredibly rewarding. I'm certain that if we had to engage in a similar competition in the future we'd be able to draw valuable lessons from this.

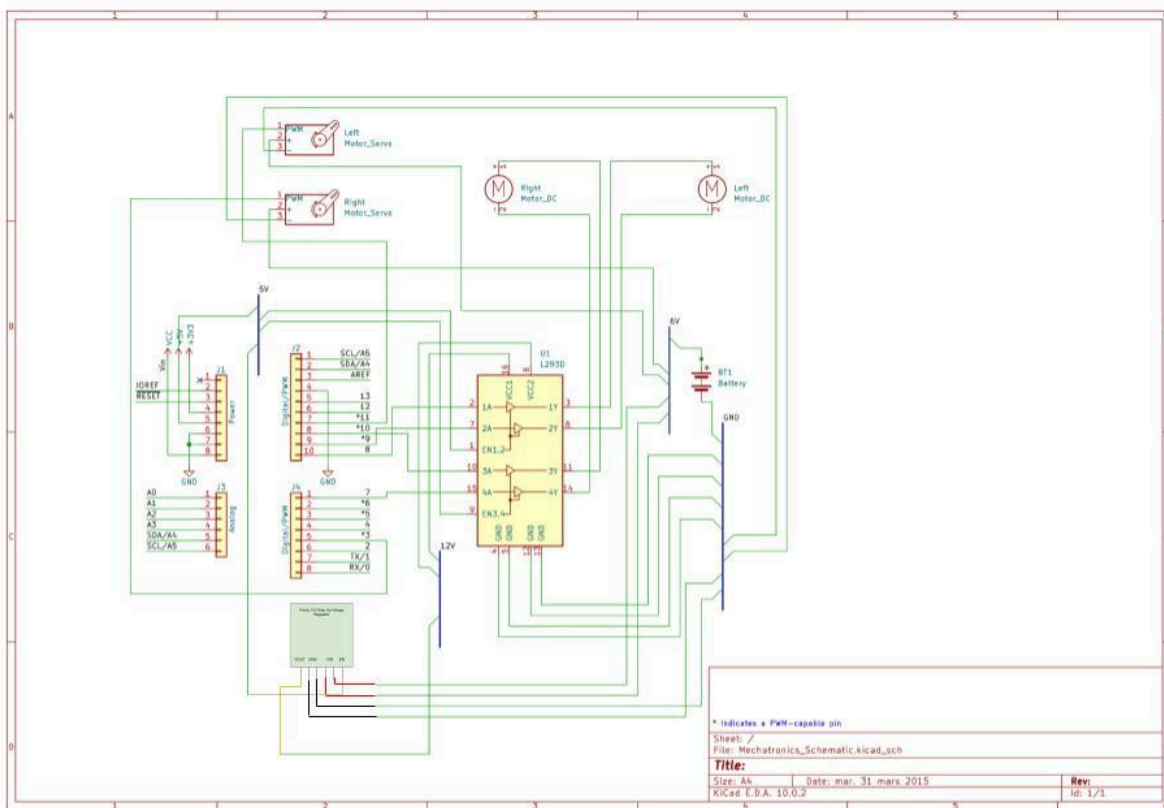
Appendix A

	A	B	C	D	E	F	G	H
1	Part	Quantity	Unit Cost	# of Teams Shared	Total	Distributor	Link	Rationale
2	90mm Wheel	2	2	n/a	4	MAE 3270		Takes wheel diameter from 66mm to 91mm. This is a 38% increase in distance per rotation
3	Acrylic Sheet	2	5	n/a	10	RPL		Base material
4	Part Count	5	0.5	n/a	2.5	RPL		
5	Cut Line Distance	40	0.05	n/a	2	RPL		
6	Acrylic Adhesive	1	9.68	3.00	3.23	McMaster Carr	https://www.mcmaster.com/	For bonding acrylic - acrylic We can split this cost with another team if we need
7	Caster Wheel 20mm	3	1.95	n/a	5.85	Adafruit	https://www.adafruit.com/	For easier pivoting and stability on arms
8	12V Step-Up Voltage Regulator	1	9.95	n/a	9.95	Pololu	https://www.pololu.com/	For more torque and speed
9								
10								
11	Total							
12	37.53							

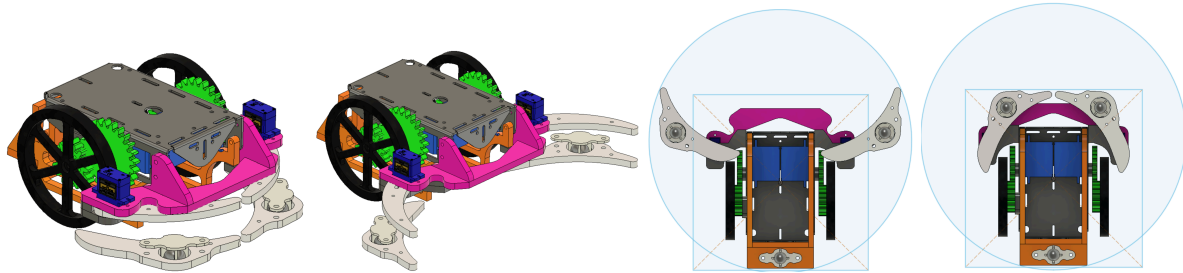


In order to reduce the cost attributed to individual parts, we joined the individual parts in the DXF as shown above, combining them into effectively “larger parts”. This reduced the total number of parts to the number of times we used the laser cutter.

Appendix B



Appendix C

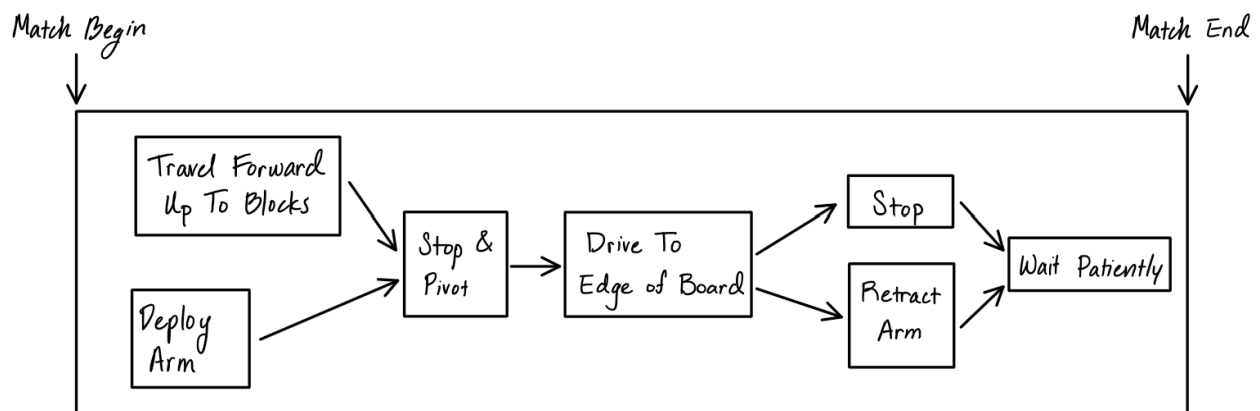


Major components not modelled: 6V Battery, Arduino Uno, 9V Battery, Wires, 12V Step-Up Board, Bolts, & Standoffs

Color v. Assembly	
Black	Wheels
Green	Gear Train
White	Arms
Pink	Front Bumper
Orange	Rear Bumper & Cage

Nothing was 3D printed so part weight is irrelevant, but we were told by TAs to estimate the total line length as about a dollar or so (considering that we cut a significant portion of both sheets, we decided to double that value).

Appendix D



The robot behavior was hard-coded under the assumption that it would be faster than other robots and therefore be able to capture the blocks before they could get there. If we can do this

reliably, it's no longer necessary to consider reacting to new situations such as being redirected towards a border after an encounter with another robot.

Appendix E

```
// Benjamin Brown, Julius Goldberg, Elijah Fesit
// Group 61, MAE 3780 Mechatronics

volatile int previousSpeed = 255;
volatile int finalSpeed = 255;
int OWS = 10;
int IWS = 10;

void open_arms(int leftorright) {

    if (leftorright == 1) {
        OCR2B = 3; // set positional servo to lower bound, opens left arm

        // CEREAL
        Serial.println("EXLA");
        _delay_ms(10);
    }

    else if (leftorright == 2) {
        OCR2A = 20; // set positional servo to upper bound, opens right arm;
        // difference from left arm code is due to servo turning in opposite
        // direction to open

        // CEREAL
        Serial.println("EXRA");
        _delay_ms(10);
    }
}

void close_arms(int leftorright) {
    if (leftorright == 1) {
        OCR2B = 20; // set positional servo to upper bound, closes left arm
        PORTD &= ~(0b00001000); // sets pin 3 low

        // CEREAL
```

```

        Serial.println("CLA");
        _delay_ms(10);
    }

    else if (leftorright == 2) {
        OCR2A = 3; // set positional servo to upper bound, closes right arm
        PORTB &= ~(0b00001000); // sets pin 11 low

        // CEREAL
        Serial.println("CRA");
        _delay_ms(10);
    }
}

void XLR8(int increment, int dt, int sperr) {
    for (((OCR1A = previous speed), (OCR1B = (previous speed - sperr)));
(OCR1A < final speed); increment) {
        PORTB &= 0b11111110; // sets pin 8 low, tells left wheel to spin
forward
        OCR1A = OCR1A + increment; // increments OCR1A until it reaches
final speed, increasing rpm of left wheel
        _delay_ms(dt); // delay to choose how fast to accelerate

        // CEREAL
        Serial.println(OCR1A); //
        Serial.println("increasing left motor speed");

        PORTD &= 0b01111111; // sets pin 7 low
        OCR1B = OCR1B + increment; // increments OCR1B, increasing rpm of
right wheel
        _delay_ms(dt); // delay

        // CEREAL
        Serial.println(OCR1B);
        Serial.println("increasing right motor speed");
    }
    previous speed = final speed;
}

void constant speed() {

```

```

// LEFT MOTOR
PORTB &= 0b11111110;    // sets pin 8 low
OCR1A = (previoussspeed); // controls PWM signal sent to pin 9. higher
value --> higher rpm

// RIGHT MOTOR
PORTD &= 0b01111111;    // sets pin 7 low
OCR1B = (previoussspeed - 20); // controls PWM signal sent to pin 10.
higher value --> higher rpm

// CEREAL

Serial.println("going forward");
}

void turnonadime(int clock) {

    if (clock == 1) { // left motor (OCR1A) forwards, right motor (OCR1B)
backwards.
        PORTB &= 0b11111110; // sets pin 8 low, spin forward
        OCR1A = 255; // sets left wheel speed to max

        PORTD |= 0b10000000; // sets pin 7 high, spin backward
        OCR1B = 0; // sets right wheel speed to max; backwards rpm logic
because pin 7 is high now

    }

    else if (clock == 2) {
        PORTB |= 0b00000001; // sets pin 8 high, spin backward
        OCR1A = 0; // sets left wheel to max speed

        PORTD &= 0b01111111; // sets pin 7 low, spin forward
        OCR1B = 255; // sets right wheel to max speed
    }
}

void turngosling(int turn_radius, int direction) {

```

```

//Calculate the speed that the outside wheel needs

PORTD &= 0b01111111; // sets pin 7 low
PORTB &= 0b11111110; // sets pin 8 low

if (direction == 1) { // turning right

    OWS = OCR1B; // sets outer wheel speed to OCR1B (generally the speed
of either wheel before this function is used)
    IWS = OWS * (turn_radius - 5 / 2) / (turn_radius + 5 / 2); // relates
inner wheel speed to outer wheel speed, no need to specify both

    OCR1A = OWS; // sets left wheel speed to outer wheel
    OCR1B = IWS; // sets right wheel speed to inner wheel speed

    // CEREAL
    Serial.println(OCR1A);
    _delay_ms(200);
    Serial.println("turning right");
    _delay_ms(200);
} else if (direction == 2) { // turning left

    OWS = OCR1A; // outer wheel speed is OCR1A
    IWS = OWS * (turn_radius - 5 / 2) / (turn_radius + 5 / 2); // sets
inner wheel speed

    OCR1B = OWS; // sets right wheel as outer wheel
    OCR1A = IWS; // sets left wheel as inner wheel

    // CEREAL
    Serial.println(OCR1A, OCR1B);
    Serial.println("turning left");
}
}

int main(void) { // setup code that only runs once

    Serial.begin(9600);
    DDRB = 0b0001111; // sets pins 8-11 as outputs
    DDRD = 0b10001000; // sets pins 3 and 7 as outputs

```



```
TCCR1A = 0b10100001; // output compare for both OCR2A/B
TCCR1B = 0b00001001; // prescaler of 1 for non-oscillating speed; found
to be best for our motors
TCCR2A = 0b10100001; // output compare for both OCR1A/B
TCCR2B = 0b00000111; // highest prescaler

// TESTING STUFF

// open_arms(1);
// open_arms(2);
// _delay_ms(300);
// _delay_ms(300);
// constantspeed();
// _delay_ms(2000);
// OCR1A = 0;
// OCR1B = 0;
// close_arms(1);
// close_arms(2);
// _delay_ms(300);
// OCR2A = 0;
// OCR2B = 0;

// turnonadime(2);

// COMPETITION TIME

open_arms(1);
open_arms(2);
_delay_ms(350);
OCR2A = 0;
OCR2B = 0;
constantspeed();
_delay_ms(1900);
turnonadime(1);
_delay_ms(680);
constantspeed();
_delay_ms(3350);

close_arms(1);
close_arms(2);
```

```
OCR1A = 0;
OCR1B = 0;
_delay_ms(400);
}
```